GRIFFITH COLLEGE

Assignment Cover Sheet

Learner Names: Briyon Benny, Joyal Joy, Ayo-oluwa Joshua Olajide

Learner Numbers: 3144036, 3147134, 3139178

Faculty: _____Computing_____

Programme: _____BSCO/H_____ Stage/Year: _____3_____

Module: _____WT_____

Study Mode: Full Time _____ Part-time _____

Lecturer Name: ____Martin Dow_____

Assignment Title: _____Assignment Two_____

Additional Relevant Information (e.g. number of pieces submitted etc.):

_____

AI in Learner Assessment Policy Indicate here applicable categories allowed for this assignment.	Category of AI Use	Allowed or not allowed
	No AI Use	
	AI for Planning	
	AI for Editing	X
	AI for Support Tasks	
	AI for Collaboration	
	Full AI Use	**(NOT ALLOWED)**

Academic Integrity Honour Code:
I submit this work in line with the principles of Academic Integrity as appearing in the Academic Integrity policy, the assessment description(s), and the relevant AI Assessment Scale in the AI in Learner Assessment

I affirm that I have not given or received any *unauthorised* help, from a person or through unauthorised content generation on this assignment, and that this work is my own.

I understand that penalties may be imposed if this assessment is in breach of the Academic Integrity and Misconduct policy.

Signature: _____ Date: ___15/04/2026_____

Please note: Learners **MUST** retain a hard/soft copy of all assignments and must have the lecturer/member of Faculty acknowledge/sign as proof of submission.

Table of Contents

Project Overview 3
Objectives 3
Features Implemented 3
3.1 User Authentication 3
3.2 Listings (CRUD) 3
3.3 Favourites 4
3.4 Sessions & Cookies 4
3.5 Validation 4
3.6 JSON API & Testing 4
3.7 Database 4
Changes from Proposal 4
Database Design 5
API Endpoints 6
Validation & Error Handling 6
Sessions & Cookies 6
Technologies Used 7
Deployment 7
Team Contributions 7
Conclusion 8

Project Overview

SafeStay is a web application designed for student-oriented accommodation listings, developed as part of Assignment 2 using server-side technologies. The platform allows property owners to create, update, and delete their own listings, while students can browse available accommodations using optional filters, view detailed information, and maintain a personalised favourites list. The interface is rendered on the server using EJS templates, and a JSON API available under /api supports automated testing (e.g., Bruno) while using the same session-based authentication as the web interface. The application addresses the need for a focused platform where students can easily find accommodation based on location, property type, and budget, while also enabling property owners to securely manage listings through role-based access control, ensuring only authorised users can perform specific actions.

Objectives

The SafeStay project delivers a fully functional server-side web application built with Node.js, Express, MongoDB, and EJS, in line with the module brief requirements. It implements secure user management, including signup, login, logout, password hashing, and role-based access control for student and owner accounts. Full CRUD functionality is provided for listings, with strict ownership and role validation to ensure that only authorised users can create, update, or delete their own listings. Session management is handled through secure server-side sessions and cookies for authentication, along with an additional non http Only cookie used for convenience features. The application also includes server-side validation and ensures consistent behaviour across both HTML form based interactions and the JSON API, supporting reliable use in both browser environments and automated testing scenarios.

Features Implemented

3.1 User Authentication

The application provides full user authentication through both HTML interfaces and a JSON API. Users can sign up using their full name, email, password, and assigned role (student or owner). They can also log in and log out securely. Passwords are hashed using bcryptjs before being stored in the database, ensuring that no plain-text passwords are saved. The system supports role-based access control, where students and owners have different permissions, particularly for listing creation and management.

3.2 Listings (CRUD)

SafeStay implements full CRUD functionality for property listings. Owners are authorised to create listings through both the web interface and API. All users can view listings, including browsing all listings with optional filters such as location, property type, and maximum price,

as well as viewing individual listings by ID. Update and delete operations are restricted so that only the original listing owner can modify or remove their listings.

3.3 Favourites

Authenticated users can toggle listings as favourites. A dedicated page is provided for users to view their saved favourite listings, accessible through both the HTML interface and a corresponding API endpoint.

3.4 Sessions & Cookies

Session management is handled using express-session, with session data stored in MongoDB via connect-mongo. The session cookie is named safestay.sid and is configured with httpOnly and sameSite=lax for security. In addition, a non-httpOnly cookie called lastLoginEmail is used to improve user experience by pre-filling the login form with the user's last used email address.

3.5 Validation

The application uses shared validation middleware for signup, login, and listing operations. For HTML requests, validation errors are re-rendered directly on the form. For API requests, errors are returned as structured JSON responses with status code 400 in the format { "errors": [...] }, while authentication and authorisation failures return 401 or 403 status codes as appropriate.

3.6 JSON API & Testing

All API routes are grouped under /api and use session-based authentication rather than JWT, ensuring consistency with the web interface. A Bruno test collection is included under bruno/SafeStay/ for structured and repeatable API testing. A /health endpoint is also provided to support deployment smoke tests.

3.7 Database

The application uses MongoDB as its database, with Mongoose models defining the structure for both users and listings, ensuring consistent schema management and data integrity.

Changes from Proposal

The original design proposed in Assignment 1 included several advanced features such as a real-time chat system, reporting functionality, and admin verification workflows.

However, these features were not implemented in this iteration of the project. The development effort was instead focused on delivering the core backend requirements specified in the assignment brief.

The following key functionalities were prioritised and successfully implemented:

- User authentication (registration, login, logout)
- Session management and cookies
- Property listing CRUD operations
- Favourites (saved listings) functionality
- Server-side validation
- JSON API for data interaction
- Deployment of the application online

Additionally, a full React-based front-end (MERN stack) was not included in this submission. Instead, the application uses server-rendered pages with EJS alongside the JSON API to demonstrate functionality.

It is worth noting that the listing data model includes an isVerified field. This was added to support future implementation of admin verification features, as originally outlined in the proposal.

Database Design

The application uses MongoDB with Mongoose to manage and structure data efficiently. The system consists of three main collections: Users, Listings, and Sessions, each designed to support the core functionality of the SafeStay platform.

The Users collection stores all registered users, including both students and property owners. Each user has a full name, a unique email address stored in lowercase, and a securely hashed password using bcrypt. In addition, each user has a role field that defines whether they are a student or an owner, with the default role set to student. The model also includes a favourites field, which is an array of ObjectIds referencing saved listings, allowing users to store preferred properties. Mongoose automatically manages the createdAt and updatedAt timestamps for each user document.

The Listings collection stores all accommodation listings created within the platform. Each listing includes a title, location, price, property type, and a detailed description, all of which are required fields. The price field is validated to ensure it has a minimum value of 1. Each listing is associated with a specific user through the owner field, which references the User collection. Additionally, the listing model includes an isVerified field, which is set to false by default and reserved for future implementation of admin verification features. Similar to the Users collection, timestamps are automatically managed for each listing.

Session management is handled using Express sessions, with session data stored in MongoDB through connect-mongo in a dedicated sessions collection. Each session stores the authentication state of a user, allowing them to remain logged in across multiple requests.

Cookies are used on the client side to store the session ID, enabling secure and persistent user sessions throughout the application

API Endpoints

The application provides both HTML routes and a JSON API. HTML routes are organised under /auth and /listings, while the JSON API is available under /api/auth and /api/listings. The general routes include a redirect from / to /listings, a /health endpoint for deployment checks, and /api which returns a short description of the API. Authentication in the HTML layer includes routes for signup, login, and logout, where users can create accounts, log in with session creation, and log out by destroying their session; during signup and login, a lastLoginEmail cookie is set or used to pre-fill the login form. Listing routes in the HTML interface allow users to view all listings with optional filters, create new listings (owners only), edit and delete listings (owner only), view individual listings, and toggle favourites, with a dedicated page for viewing saved favourites. The JSON API mirrors this functionality under /api, including endpoints for authentication, listing management, and favourites, and uses the same session-based authentication system rather than JWT, ensuring consistency between web and API usage.

Validation & Error Handling

Validation is implemented consistently across both HTML and API layers. For signup, the system enforces a minimum full name length of two characters, valid email format, password length of at least six characters, and a valid role of either student or owner, while duplicate emails return appropriate conflict errors. Login validation checks email format and password requirements, and invalid credentials return a generic error message. Listing validation ensures that the title is at least three characters long, location and property type are required, price must be a number greater than zero, and description must be at least ten characters long. In the API, validation errors return a 400 status code with a structured { errors: [...] } response, authentication failures return 401, and unauthorized actions such as a student attempting to create a listing or a non-owner modifying a listing return 403 errors.

Sessions & Cookies

User authentication is managed through server-side sessions where, after login or signup, a minimal user object containing id, name, email, and role is stored in req.session.user. The session ID is stored in a secure httpOnly cookie named safestay.sid, which is persisted in MongoDB using connect-mongo so that sessions remain active even after server restarts. In addition, a non-httpOnly cookie called lastLoginEmail is used purely for convenience to pre-fill the login form. Role-based access control is enforced through sessions, where only users with the owner role can create, update, or delete listings, while students are restricted to browsing and managing favourites.

Technologies Used

The SafeStay application is built using a range of modern web development technologies to support server-side functionality, data management, and user interaction. The backend is powered by Node.js, which provides a JavaScript runtime environment for building scalable server-side applications. Express (version 5) is used as the main web framework to handle HTTP routing and middleware, enabling efficient request and response handling throughout the application.

MongoDB is used as the primary database to store user data, property listings, and session information in a flexible document-based format. Mongoose is integrated as an Object Data Modelling (ODM) library to define schemas and manage interactions with the database, specifically for the User and Listing models. For security, bcryptjs is used to hash user passwords before storing them, ensuring that sensitive data is protected.

User authentication and session management are implemented using express-session, which maintains server-side sessions, while connect-mongo is used to store session data in MongoDB for persistence. Additionally, cookie-parser is used to read and write cookies, such as storing the lastLoginEmail to improve user experience during login.

The application uses EJS (Embedded JavaScript) as the templating engine to render dynamic HTML pages on the server side. To support RESTful operations through HTML forms, method-override is used to allow PUT and DELETE requests via a special _method field. Environment variables are managed using dotenv, which loads configuration settings from a .env file.

Finally, Bruno is used as an API testing tool to verify and test the application's endpoints during development, although it is not part of the runtime dependencies.

Deployment

The SafeStay application is designed to be deployed as a web service on platforms such as Render. Deployment involves installing dependencies using npm install and starting the application with npm start. For production, essential environment variables such as MONGODB_URI and SESSION_SECRET must be configured securely. A health check endpoint (GET /health) is available to verify that the application is running correctly after deployment. The application also provides a JSON API accessible from the same base URL as the website, and tools like Bruno can be configured with the production base URL to test API endpoints remotely.

Team Contributions

The project was developed by a team of three members, each contributing to specific areas of the SafeStay application to ensure full coverage of backend, frontend, API, and deployment functionality.

Briyon Benny (3144036) was responsible for implementing listing CRUD functionality in the HTML routes, enforcing role-based access control (owner/student permissions), managing session user synchronization from MongoDB, and handling listing view, edit, and delete behaviour. He also contributed to local integration testing of listing features.

Joyal Joy (3147134) worked on the authentication system, including user signup, login, logout, password hashing, and session and cookie management. He also implemented shared validation logic, developed the JSON authentication API (/api/auth), and built the authentication-related views for the application.

Ayo-oluwa Joshua Olajide (3139178) focused on the frontend structure and API integration, including EJS templates and layout design for listings and favourites, UI filters and form usability improvements, and the JSON listings API (/api/listings). Additionally, he prepared the Bruno API testing collection, deployment documentation (including Render setup, environment variables, and /health endpoint), and overall submission documentation.

Conclusion

This submission presents a coherent SafeStay prototype featuring authenticated users, role-based listing management, favourites functionality, persistent sessions, shared validation for both HTML and API interactions, and a testable JSON interface. While the current implementation meets the core requirements, several enhancements could be considered as future work, including admin verification of listings, integrated chat or messaging features, reporting mechanisms, and the development of a separate single-page application (SPA) client. These improvements were part of the original design but were deferred for this stage of the project.